

Permutation Tests for ANOVA and Regression - Examples in R

B.A. Hartlaub and A.S. Wagaman

Permutation Tests

In a permutation test, you investigate how unusual your observed statistic is based on considering other arrangements, *permutations*, of the data, and the statistic (or summary measure) of interest is computed for each permutation, where permutations are designed to match a setting where the null hypothesis of interest is true. The challenge is figuring out what you need to permute to make the null hypothesis true - and that changes based on the analysis you are doing.

Previously, when comparing centers for two groups, we explored permutation tests of whether the centers of the two groups were equal by permuting the group labels.

Now, we want to extend the methodology to the one-way layout (ANOVA) and regression settings. Note that in regression, we will look at some very simple approaches, and if you get more complicated than what is shown here, you should confer with your instructor to confirm your approach is statistically valid.

The process for permutation tests in these settings is similar to what we saw before. We try to construct a null distribution for some value of interest - either a test statistic or slope coefficient, usually. Then, we can assess how unusual the observed value is based on the simulated null distribution. Whether you “shuffle” the predictor or response (or something else) depends on what you are testing. For regression, we often focus on one predictor (challenges with this approach, described below), while in ANOVA, the group variable may be shuffled or more complicated designs might be necessary (for multi-way settings).

The rationale behind using these methods is that they are less sensitive to the conditions for the parametric and nonparametric test procedures we’ve seen. They require that you had random assignment to groups (for ANOVA) or of the values (regression) (or believe randomization is reasonable), and that observations are independent.

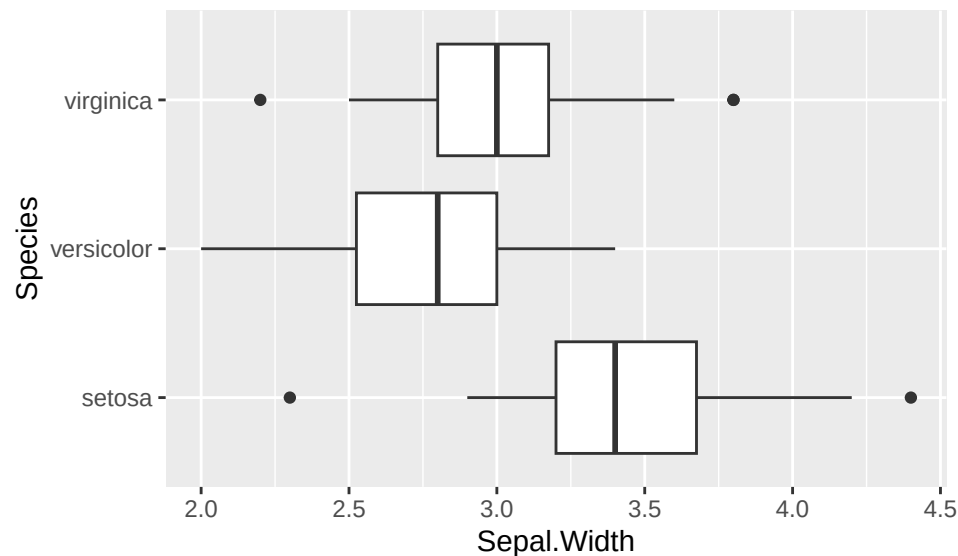
Permutation Tests for One-Way Layout (ANOVA)

In order to perform a permutation test, we have to consider what we would be permuting. Here, the idea is we are comparing centers between our k groups, and we have a statistic (F for ANOVA or H for Kruskal-Wallis) that summarizes how different the centers are.

If the null hypothesis is true, all observations come from the same population, so our statistic was just determined by randomly dividing the observations into their k groups. If we reshuffle the group labels, we can simulate this process. The idea is that if our data indicates a significant difference in centers is present, the test statistic we observed will be extreme compared to those generated via the reshuffling process. All we have to do is obtain many statistics from the reshuffled situation, and compare our test statistic with them. You do have to pick which test statistic you want to save - the F from ANOVA or the Kruskal-Wallis H .

We demo this on the iris data, as a simple example. Suppose we are considering whether Sepal.Width has different centers across the three iris species. What do you think the test will find based on some exploratory analysis?

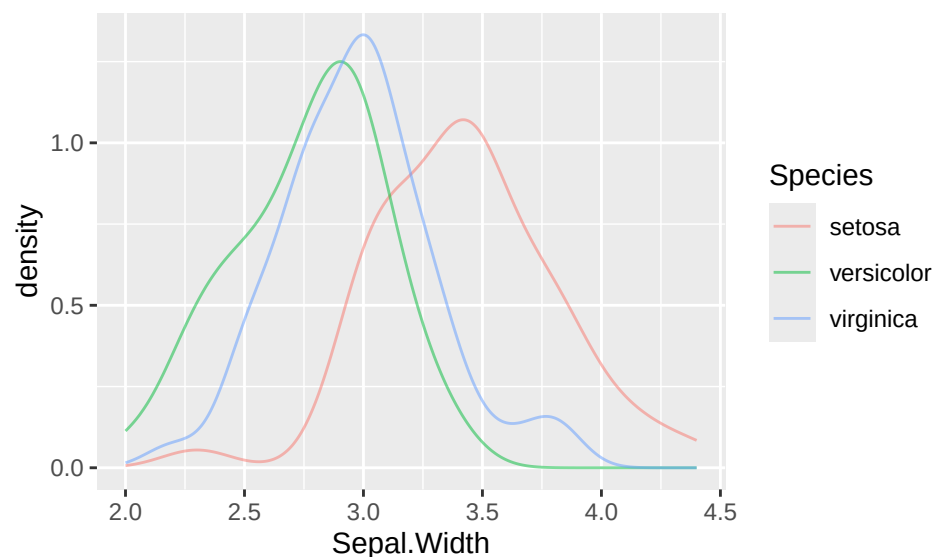
```
data(iris)
gf_boxplot(Species ~ Sepal.Width, data = iris)
```



```
favstats(Sepal.Width ~ Species, data = iris)
```

```
##      Species min    Q1 median    Q3 max  mean      sd  n missing
## 1   setosa  2.3 3.200    3.4 3.675 4.4 3.428 0.3790644 50      0
## 2 versicolor 2.0 2.525    2.8 3.000 3.4 2.770 0.3137983 50      0
## 3  virginica 2.2 2.800    3.0 3.175 3.8 2.974 0.3224966 50      0
```

```
gf_dens(~ Sepal.Width, color = ~ Species, data = iris)
```

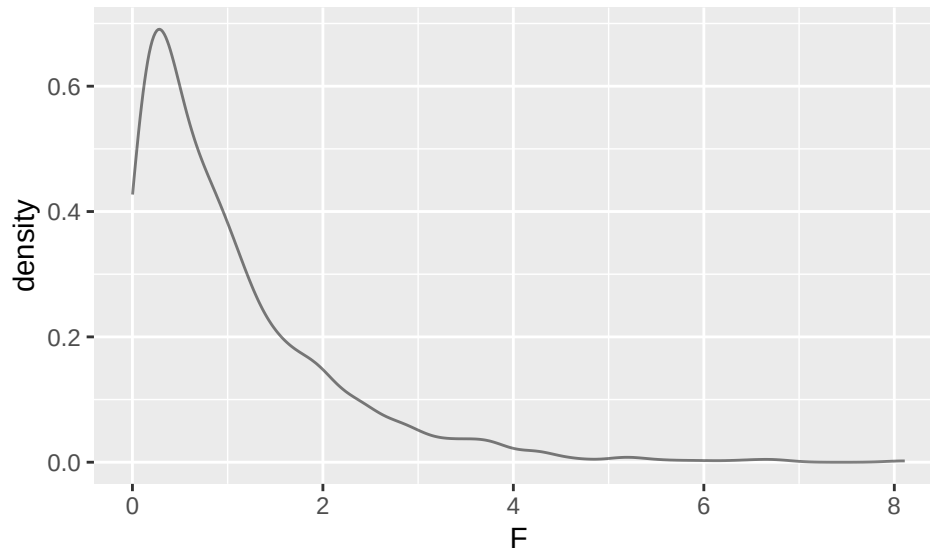


Here's how to run one of the permutation tests. This chunk demonstrates saving both the F and H values - but separately (i.e. not on the same randomly permuted data sets). The `do` function is in *mosaic*, and it works for certain functions, but not for our nonparametric functions, so those have to be run through loops. (Note, we could not get this to work for `pKW`, so it uses `kruskal.test`.) In this case, we do the runs separately, but you could envision a procedure where you generate the 1000 data sets and compute the statistics so they are on the same data (it takes a bit more work).

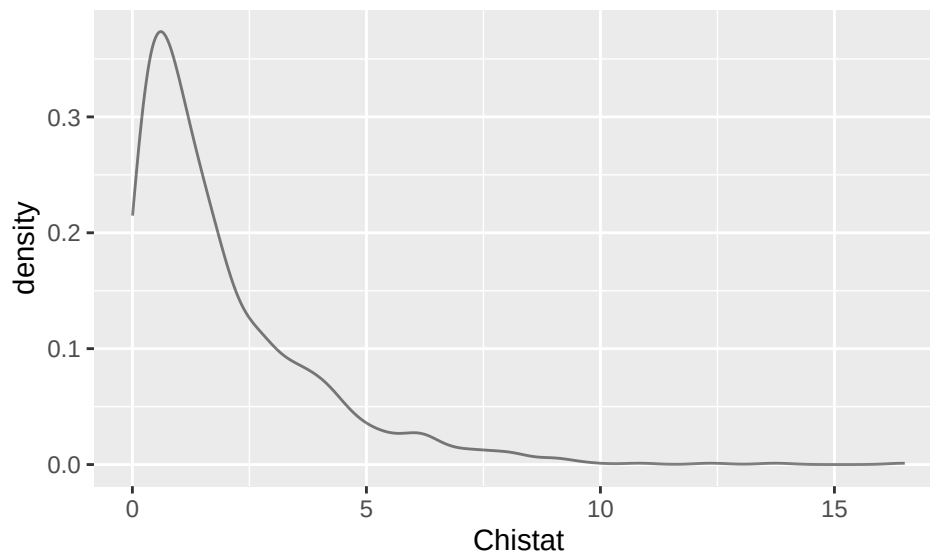
```
set.seed(225)
Fstat <- do(1000)*(anova(lm(Sepal.Width ~ shuffle(Species), data = iris)))
```

```
gf_dens(~ F, data = Fstat)
```

```
## Warning: Removed 1000 rows containing non-finite outside the scale range
## (`stat_density()`).
```



```
Chistat <- rep(0, 1000)
for(i in 1:1000){
  Chistat[i] = kruskal.test(Sepal.Width ~ shuffle(Species), data = iris)$statistic}
gf_dens(~ Chistat)
```



What does each density plot show you? (That is, what are the values in it?) What would you put as an appropriate title for the plot?

For each of these, we need to compare the observed F and H to the distributions we obtained. So, what values did we have originally? (Usually you'd get these before doing the permutation procedure.)

```
anova(lm(Sepal.Width ~ Species, data = iris))$F # parametric ANOVA F
```

```
## [1] 49.16004      NA
kruskal.test(Sepal.Width ~ Species, data = iris)$statistic # chi-squared approx for KW

## Kruskal-Wallis chi-squared
##                63.57115

set.seed(225)
with(iris, pKW(x = Sepal.Width, g = Species))$obs.stat # exact H

## [1] 63.03125
```

How extreme would you say the observed F and chi-squared values were?

In either case, it's pretty clear that the observed values are VERY extreme compared to the permuted data test statistic values. This is evidence that we do have significant differences in centers between the groups (at least somewhere).

Permutation (Randomization) Methods in Regression

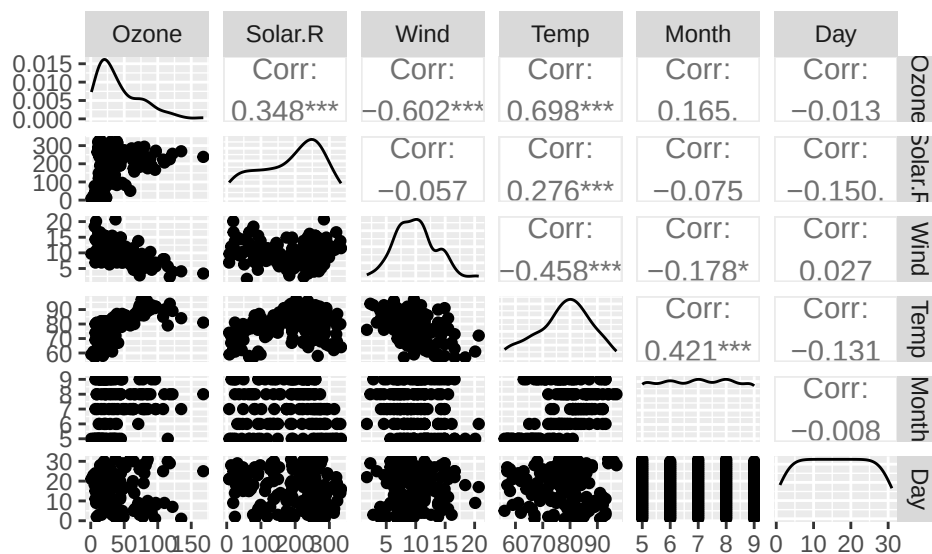
These methods can also be used in regression. Here, the idea is that we can assess whether the linear relationship (or correlation equivalently) in simple linear regression is really present if we “destroy” the current relationship between X and Y by permuting X and see how unusual our test statistic or slope coefficient is compared to test statistics or coefficients generated from permuted data. If there are multiple predictors, by permuting just X , you leave the relationship that Y has with the other predictors alone. This allows us to see if we need X in a model that has the other predictors already. On the other hand, if you want to see if there is a relationship between Y and any of the other X s, you might permute Y . When you are in a multiple linear regression, it's a little more complicated than just shuffling the X whose coefficient you are interested in - better procedures involve fitting main effects, getting residuals and working with those (you need to see if the part of Y that is not explained by the other predictors can be explained by the part of that X the other predictors can't substitute for). However, we'll use these examples as reasonable starting points. You can explore more depending on what you want to investigate later.

To look at some regressions, we'll use the airquality data set in R.

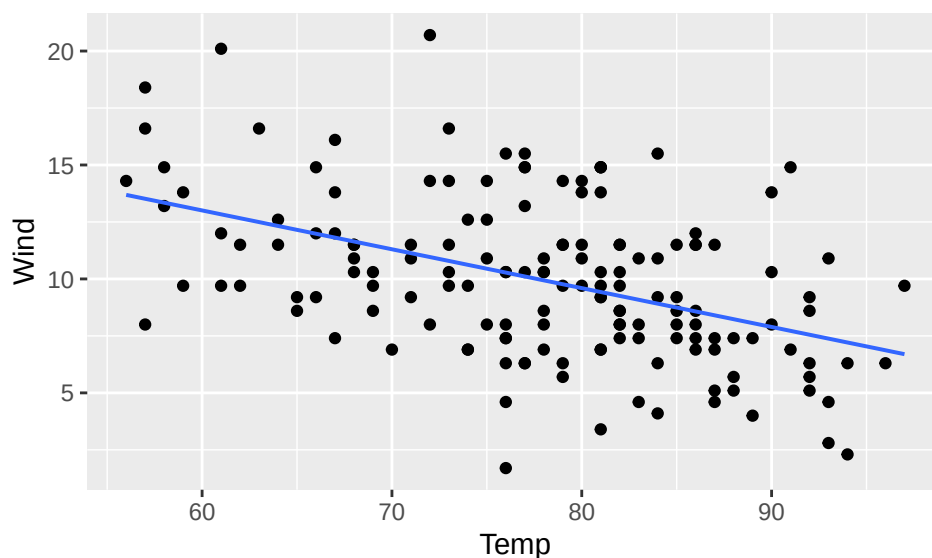
```
data(airquality)
```

If we look at the airquality data set, we can see some linear relationships that we might want to assess. For example, the relationship between Wind and Temperature looks linear but not very strong - is it really significant?

```
ggpairs(airquality)
```



```
gf_point(Wind ~ Temp, data = airquality) %>% gf_lm()
```



```
model <- lm(Wind ~ Temp, data = airquality)
msummary(model)
```

```
##           Estimate Std. Error t value Pr(>|t|)
## (Intercept) 23.23369    2.11239  10.999 < 2e-16 ***
## Temp       -0.17046    0.02693  -6.331 2.64e-09 ***
##
## Residual standard error: 3.142 on 151 degrees of freedom
## Multiple R-squared:  0.2098, Adjusted R-squared:  0.2045
## F-statistic: 40.08 on 1 and 151 DF,  p-value: 2.642e-09
# using rfit, but could use theil, but it has issues with ties among X's
modelnp <- rfit(Wind ~ Temp, data = airquality)
summary(modelnp)
```

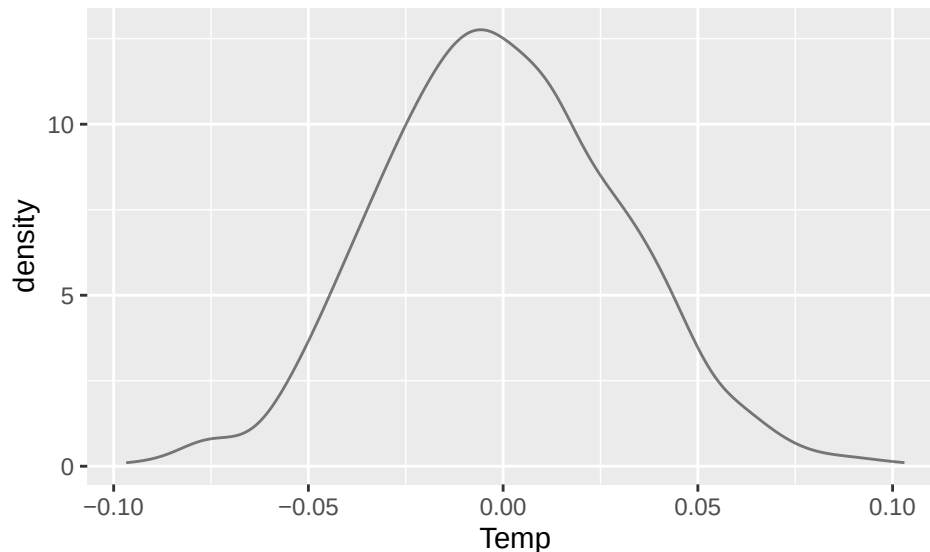
```
## Call:
```

```
## rfit.default(formula = Wind ~ Temp, data = airquality)
##
## Coefficients:
##             Estimate Std. Error t.value    p.value
## (Intercept) 23.000000    2.066333 11.1308 < 2.2e-16 ***
## Temp        -0.170370    0.026313 -6.4746 1.259e-09 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Multiple R-squared (Robust): 0.1992087
## Reduction in Dispersion Test: 37.5635 p-value: 0
```

The results from parametric and nonparametric regression indicate that the relationship between Wind and Temperature is significant (though we didn't check conditions yet). Let's see if our permutation test agrees.

First, with the parametric slope coefficient (we save these and see if they differ from 0):

```
set.seed(225)
tstat <- do(1000)*(lm(Wind ~ shuffle(Temp), data = airquality))
gf_dens(~ Temp, data = tstat)
```



```
model$coef # to remind us of original slope
```

```
## (Intercept)      Temp
## 23.2336881 -0.1704644
```

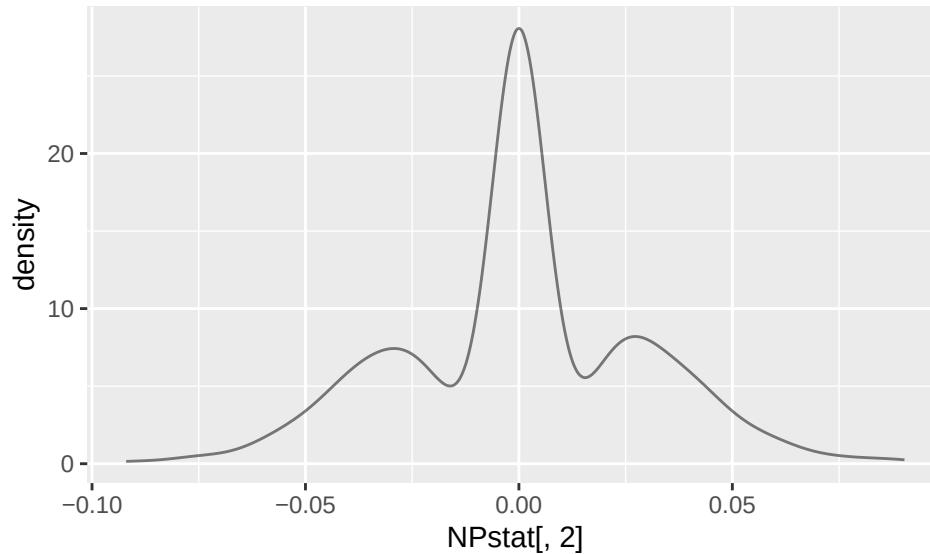
What is displayed in this density plot? What title would you give it?

Our observed slope coefficient was -0.17, which is off the lower end of this plot. Thus, we have extreme results compared to the permuted slope coefficients, confirming that there is evidence the relationship is significant.

For the nonparametric slope coefficient, once again we can't use *do*, and need to use a loop (or something similar).

```
reps <- 1000
# you could rewrite this to grab only the desired slope
temp <- rep(0, 2*reps) # use double values because it stores intercept and slope
NPstat <- matrix(temp, nrow = reps)
```

```
for(i in 1:1000){
  NPstat[i,] <- (rfit(Wind ~ shuffle(Temp), data = airquality))$coefficients}
# need 2nd entry to get slopes (first is intercept)
gf_dens(~ NPstat[, 2]) # to plot the permuted slope coefficients
```

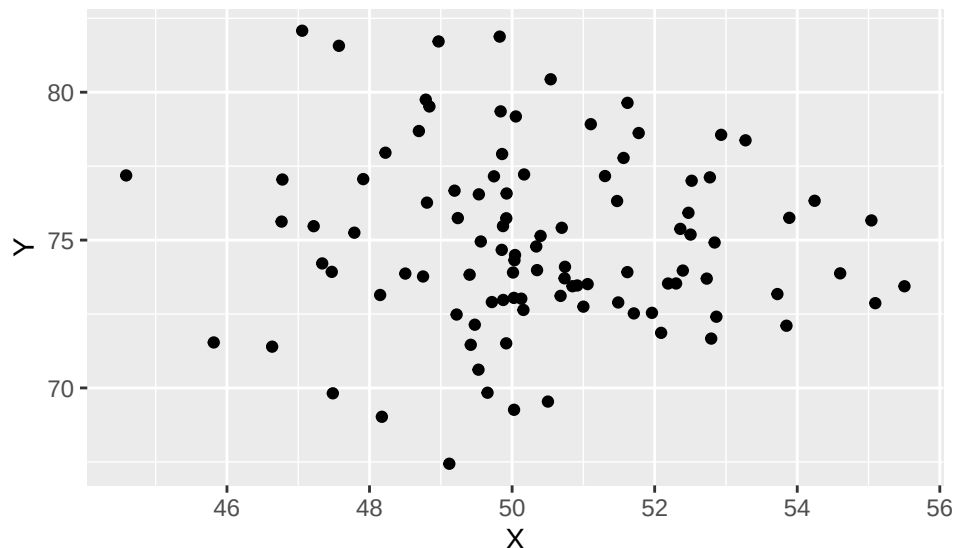


We can see that the nonparametric slope estimate from rfit was -0.17, basically the same the parametric estimate, and this is again extreme considering the distribution of the permuted slope coefficients.

An Example where the relationship is NOT significant

In this example, we create X and Y with no linear relationship whatsoever, to demonstrate what happens when there is no linear relationship present.

```
set.seed(225)
X <- rnorm(100, 50, 2)
Y <- rnorm(100, 75, 3)
gf_point(Y ~ X)
```



```
lm(Y ~ X)
```

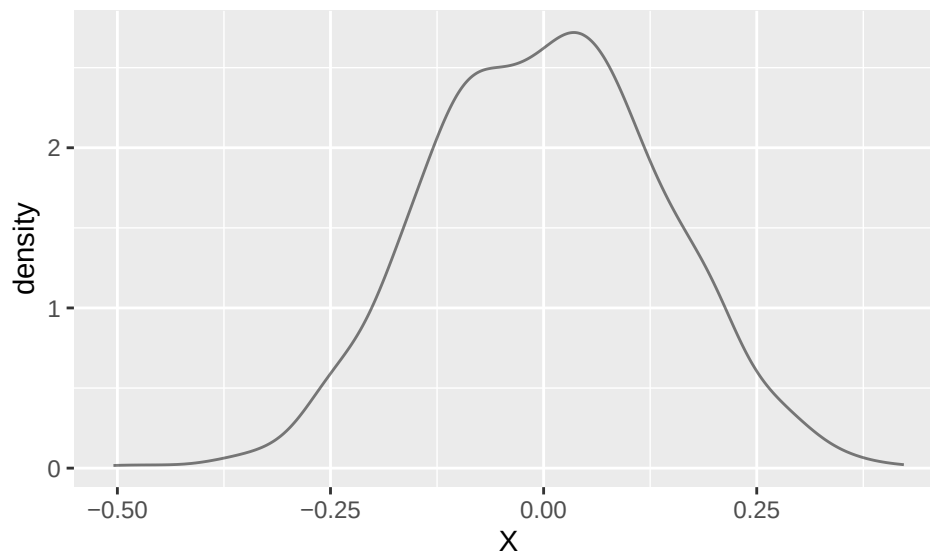
```
##  
## Call:  
## lm(formula = Y ~ X)  
##  
## Coefficients:  
## (Intercept)          X  
##    79.72893    -0.09568
```

```
rfit(Y ~ X)
```

```
## Call:  
## rfit.default(formula = Y ~ X)  
##  
## Coefficients:  
## (Intercept)          X  
## 79.16794558 -0.09514981
```

The coefficient of X from these models should be near the middle of the density plot we generate using the permutation test. Let's find out if that happens!

```
set.seed(225)  
tstat <- do(1000)*(lm(Y ~ shuffle(X)))  
gf_dens(~ X, data = tstat)
```



For this run (i.e. with the seed), our result was towards the negative side but was not too far into the tail (results will differ if you change the seed because everyone is generating different X's and Y's.)

Remember if you want an empirical p-value, you need to compute the tail probability.

```
pdata(tstat$X, -0.09515)
```

```
## [1] 0.25
```

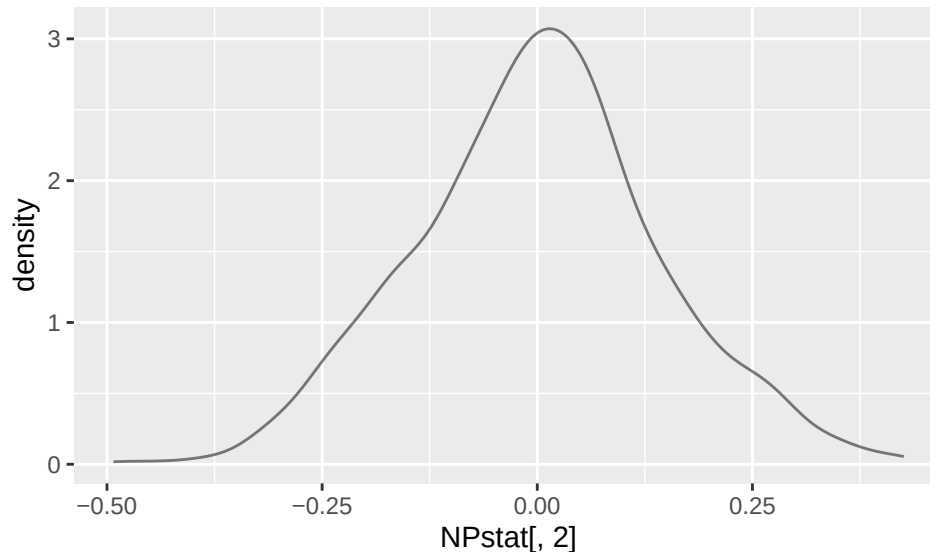
This large p-value would suggest the relationship is not significant.

And for the nonparametric test, we run a loop again.


```

reps <- 1000
NPstat <- matrix(rep(0,2*reps), nrow = reps)
for(i in 1:1000){
  NPstat[i, ] <- (rfit(Y ~ shuffle(X)))$coefficients}
gf_dens(~ NPstat[, 2]) #to plot slopes

```



We had a similar result here. This is what will happen if the relationship in question is not statistically significant. We can see the observed slope coefficient of -0.0957 is near the center of the density plot, not far into the tails.

Multiple Linear Regression

There are two usual procedures you'd be interested in here. You can permute Y to assess if it has a relationship with any of the predictors, or you can be focused on a particular predictor, and whether you need it in the model (this can work even if you have a categorical predictor of interest, you just have to think about what test statistic to use).

To do the latter test, we just shuffle the predictor we are interested in (again, there are better approaches if you want to really get into this). Here's an example, where we again focus on the coefficients of the two predictors.

```

mymod <- lm(Petal.Width ~ Petal.Length + Sepal.Length, data = iris)
msummary(mymod)

```

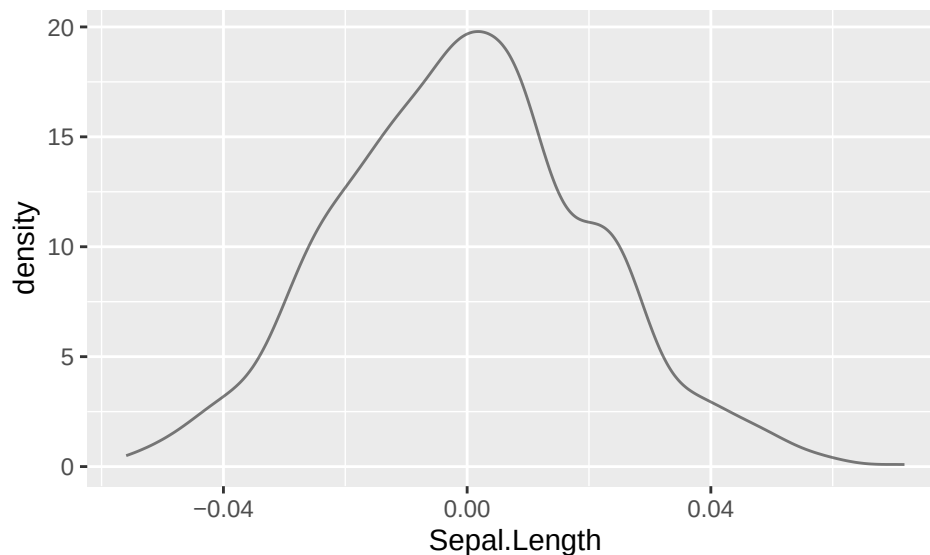
```

##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) -0.008996   0.182097  -0.049   0.9607
## Petal.Length  0.449376   0.019365  23.205 <2e-16 ***
## Sepal.Length -0.082218   0.041283  -1.992   0.0483 *
##
## Residual standard error: 0.2044 on 147 degrees of freedom
## Multiple R-squared:  0.929, Adjusted R-squared:  0.9281
## F-statistic: 962.1 on 2 and 147 DF, p-value: < 2.2e-16

```

Here, we see that Sepal.Length is barely significant. If there are issues with the conditions, we may want to know if that's really a significant predictor.

```
set.seed(225)
tstat <- do(1000)*(lm(Petal.Width ~ Petal.Length + shuffle(Sepal.Length), data = iris))
gf_dens(~ Sepal.Length, data = tstat)
```



```
pdata(tstat$Sepal.Length, -0.0822)
```

```
## [1] 0
```

What does this density plot show? What title would you give it?

What do you conclude?

There are no t statistics below -0.0822 in our simulation, so we would reject the null hypothesis and conclude that sepal length is a significant predictor based on our simulation.

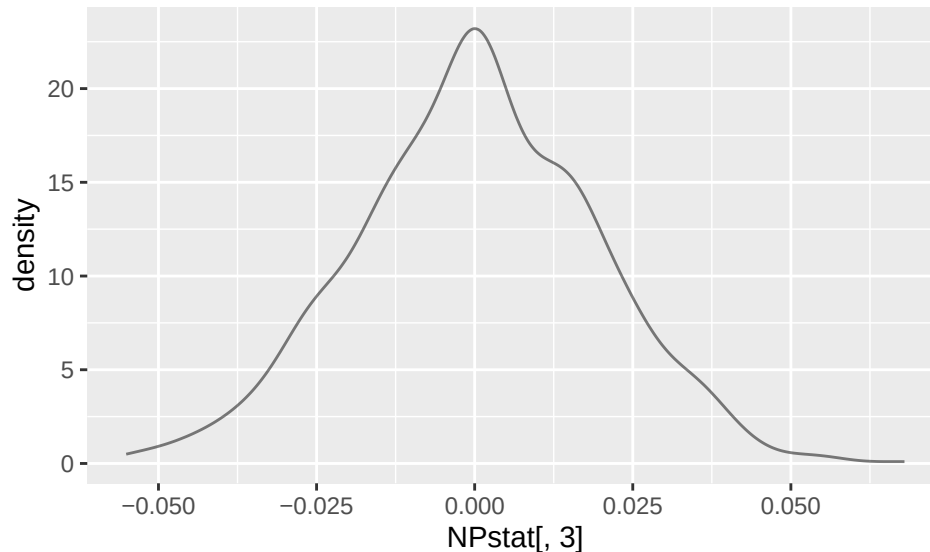
And for the nonparametric fit with rfit, we have:

```
npmod <- rfit(Petal.Width ~ Petal.Length + Sepal.Length, data = iris)
summary(npmod)
```

```
## Call:
## rfit.default(formula = Petal.Width ~ Petal.Length + Sepal.Length,
## data = iris)
##
## Coefficients:
## Estimate Std. Error t.value p.value
## (Intercept) -0.010034 0.174368 -0.0575 0.9542
## Petal.Length 0.442212 0.018508 23.8930 <2e-16 ***
## Sepal.Length -0.082919 0.039456 -2.1015 0.0373 *
## ---
## Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Multiple R-squared (Robust): 0.8529339
## Reduction in Dispersion Test: 426.2754 p-value: 0
```

```
set.seed(430)
reps <- 1000
# need 3 times rep because it stores intercept and two slopes
```

```
# you could adjust this so it only saves the one you are interested in, 3rd entry here
NPstat <- matrix(rep(0,3*reps), nrow = reps)
for(i in 1:1000){
  NPstat[i, ] <- (rfit(Petal.Width ~ Petal.Length + shuffle(Sepal.Length),
    data = iris))$coefficients}
gf_dens(~ NPstat[, 3]) # to plot for slope of interest
```



```
pdata(NPstat[, 3], -0.8291)
```

```
## [1] 0
```

What does this density plot show? What title would you give it?

What do you conclude?

Again, there are no t statistics below the estimated coefficient (-0.8291) of sepal length from rfit, so we conclude that sepal length is a significant predictor based on our simulation.

Data Sources

Data sets in these examples are included with the R software.

Airquality data. Available in R with `data(airquality)`. Help references Chambers, J. M., Cleveland, W. S., Kleiner, B. and Tukey, P. A. (1983) Graphical Methods for Data Analysis. Belmont, CA: Wadsworth.

Iris data. Available in R with `data(iris)`. Original sources referenced in R help:

Fisher, R. A. (1936) The use of multiple measurements in taxonomic problems. *Annals of Eugenics*, 7, Part II, 179–188. doi:10.1111/j.1469-1809.1936.tb02137.x.

Anderson, Edgar (1935). The irises of the Gaspé Peninsula, *Bulletin of the American Iris Society*, 59, 2–5.

One example uses simulated data.

Standard Reference

These materials were created for use in an undergraduate nonparametrics course. Where provided, textbook references refer to *Nonparametric Statistical Methods* (3rd Edition) by Hollander, Wolfe, and Chicken. Notation is chosen to be consistent with that text. However, materials may be used independently of the textbook.

License

This work is protected by creative commons license CC BY-NC-SA.